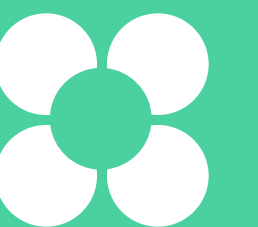
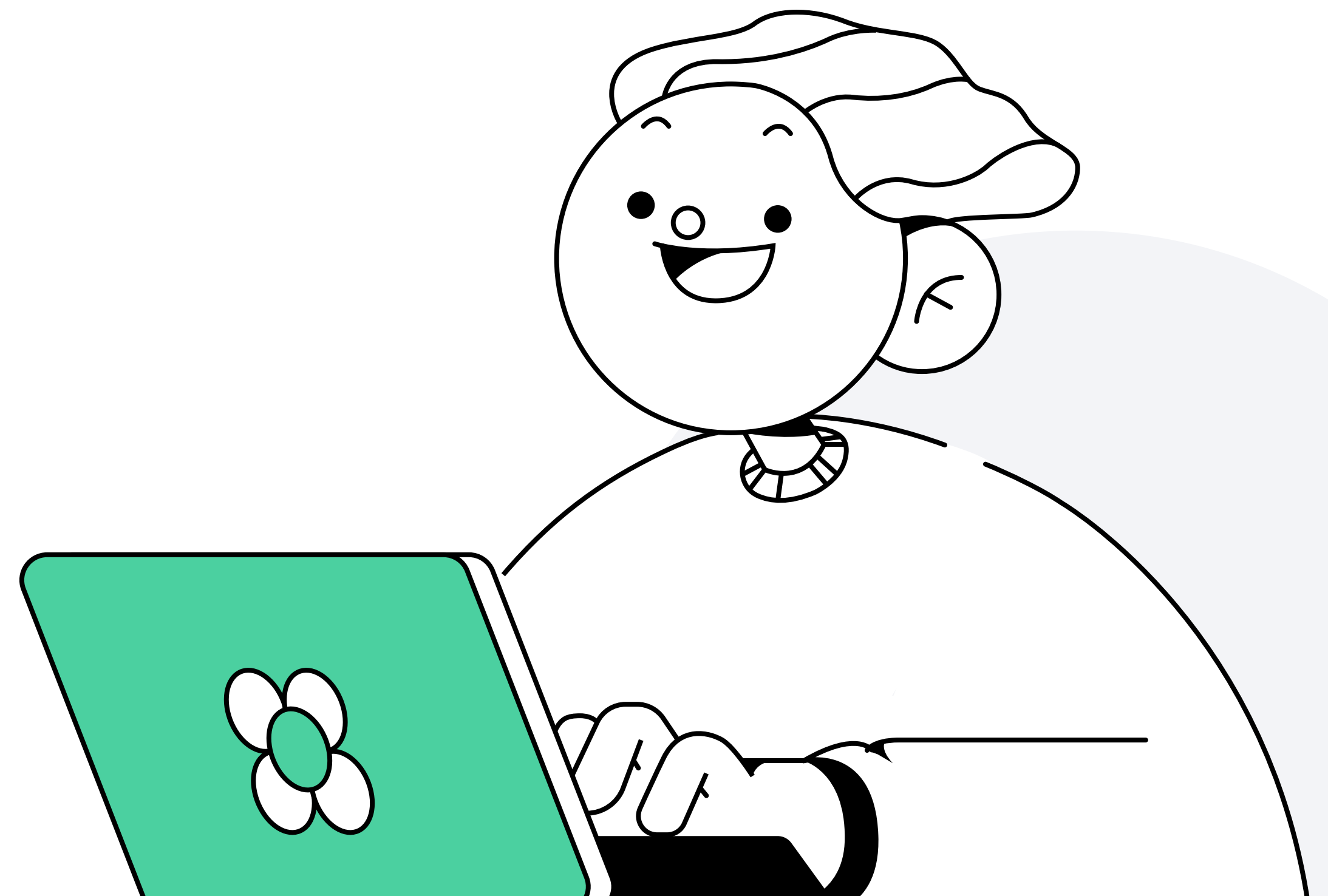


Модификаторы доступа, наследование



План занятия

- 1 Инкапсуляция
- 2 Наследование полей и методов
- 3 Наследование конструкторов



Инкапсуляция



Объектно-ориентированное программирование

Java — язык объектно-ориентированного программирования, т. е. программа состоит из различных объектов, которые взаимодействуют между собой через вызов команд и обращения к их полям.

Поэтому объекты, которые мы создаём, должны быть прописаны грамотно: так, чтобы вероятность их некорректной работы стремилась к нулю

Инкапсуляция

Рассмотрим пример:

```
public class Person {  
    public String name;  
    public int age;  
  
    public boolean isToYoung() {  
        if (age < 18) {  
            return true;  
        } else {  
            return false;  
        }  
    }  
}
```

Инкапсуляция

Возьмём поле `age`, которое содержит возраст человека. Если другой класс имеет прямой доступ к этому полю, есть вероятность, что в процессе работы программы ему будет передано некорректное значение: **`person.age = -8;`**

Мы знаем, что возраст человека не может быть отрицательным, поэтому подобное изменение данных нежелательно.

Рекомендуется как можно больше ограничивать доступ к данным, чтобы защитить их от нежелательного доступа извне: как для получения значения, так и для его изменения

Инкапсуляция

Использование различных модификаторов гарантирует, что данные не будут искажены или изменены ненадлежащим образом. Подобное сокрытие данных внутри некоторой области видимости называется **инкапсуляцией**.

Иными словами, **инкапсуляция** — это механизм обёртывания данных и кода, работающего с данными, в одно целое.

В инкапсуляции переменные класса будут скрыты от других классов, и доступ к ним может быть получен только с помощью метода их текущего класса

Использование инкапсуляции

**Переменные состояния объекта скрыты от внешнего мира.
Изменение состояния объекта (его переменных) возможно только с помощью его методов (операций).**

Такими методами выступают геттеры и сеттеры. Они настолько часто используются, что были вынесены в отдельную категорию.

Геттер — от англ. «get», «получать» — это метод, с помощью которого мы получаем значение переменной, т. е. её читаем.

Сеттер — от англ. «set», «устанавливать» — это метод, с помощью которого мы меняем, или задаём значение переменной

Использование инкапсуляции

Создадим метод, задачей которого будет менять возраст.

Чтобы защитить нашу программу от ошибки, установим условие, при котором она будет выполняться, используя метод **setAge**. Если условие не соблюдается, то программа просто выйдет, ничего не изменив, — early exit.

Также создадим специальный метод, с помощью которого мы просим человека дать нам свой возраст — **getAge**

```
public class Person {  
    public String name;  
    public int age;  
    public void setAge(int age) {  
        if (age < 0 || age > 200) {  
            return;  
        }  
        this.age = age;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

Использование инкапсуляции

Чтобы метод работал, создадим команду, которая будет просить установить возраст объекту.

Программа выведет на экран только корректное значение, подходящее заданным условиям

```
public static void main(String[] args) {  
    Person person = new Person();  
    person.setAge(40);  
    person.setAge(-8);  
    System.out.println(person.getAge());  
} //в консоль будет выведено 40
```


Модификаторы доступа

В Java используются следующие модификаторы доступа:

- **public:** публичный, общедоступный класс или член класса. Поля и методы, объявленные с модификатором `public`, видны другим классам из текущего пакета и из внешних пакетов
- **private:** закрытый класс или член класса. Противоположность модификатору `public`. Доступен только из кода в том же классе
- **protected:** такой класс или член класса доступен из любого места в текущем классе, пакете или в классах-наследниках, даже если они находятся в других пакетах
- **default:** модификатор по умолчанию. Когда мы не пишем модификатор доступа, он по умолчанию имеет значение `default`. Такие поля или методы видны всем классам в текущем пакете

Наследование полей и методов

Наследование полей: пример

Имеется класс Person, описывающий человека:

```
public class Person {  
    public String name;  
    public int age;  
    public void setAge(int age) {  
        if (age < 0 || age > 200) {  
            return;  
        }  
        this.age = age;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

Наследование полей: пример

Необходимо добавить класс для певцов — **Singer**.

Этот класс реализует тот же функционал, что и класс **Person**, так как певец — это также и человек, обладающий всеми характеристиками человека (имя, возраст). Поэтому было бы рационально сделать класс **Singer** производным (наследником, подклассом) от класса **Person**, который, в свою очередь, называется базовым классом, родителем или суперклассом:

```
class Singer extends Person {  
}
```

Наследование полей: пример

Чтобы объявить один класс наследником другого, надо использовать после имени класса-наследника ключевое слово **extends**, после которого идёт имя базового класса. Для класса **Singer** базовым является **Person**, и поэтому класс **Singer** наследует все поля и методы, которые есть в нём.

Наследование (англ. inheritance) — это механизм, позволяющий создавать классы на основе другого класса

Объектно-ориентированное программирование

Один из ключевых аспектов объектно-ориентированного программирования — это наследование.

С помощью наследования можно расширить функционал уже имеющихся классов за счёт добавления нового функционала или изменения старого. Так, один класс может наследовать характеристики другого — его методы и переменные

Наследование полей

- Класс, являющийся основой, называют базовым, суперклассом, родительским
- Класс, который создают, называют потомком, наследником или производным классом

Сама Java тоже использует механизм наследования. Например, все классы языка наследуют класс **Object**

Наследование методов

Производный класс имеет доступ ко всем методам и полям базового класса (даже если базовый класс находится в другом пакете) кроме тех, которые определены с модификатором **private**. При этом производный класс также может добавлять свои поля и методы:

```
public class Singer extends Person {  
    public void sing(String verse) {  
        System.out.println("Я пою: " + verse);  
    }  
}
```

Добавление полей и методов в производный класс никак не влияет на базовый

Наследование методов

Также в производном классе можно модифицировать поля и методы базового класса:

```
public class Singer extends Person {  
    public void sing(String verse) {  
        System.out.println("Я пою: " + verse);  
    }  
  
    @Override  
    public void happyBirthday(){  
        age++;  
        System.out.println("Я пою себе с др!");  
    }  
}
```

Наследование методов

Перед модифицированным методом указывается аннотация **@Override**. Эта аннотация не обязательна, но она помогает избежать ошибок, убеждая Java, что мы точно меняем версию уже существующего метода, а не создаём новый.

При переопределении метода он должен иметь уровень доступа не меньше, чем в базовом классе.

Например, если в базовом классе метод имеет модификатор **public**, то и в производном классе метод должен иметь модификатор **public**

Правила наследования

→ **Правило 1.** Наследоваться можно только от одного класса.

Java не поддерживает наследование нескольких классов. Один класс — один родитель.

Обратите внимание — нельзя наследовать самого себя

→ **Правило 2.** Наследуется всё, кроме приватных переменных и методов.

Все методы и переменные, помеченные модификатором `private`, недоступны классу-наследнику

→ **Правило 3.** Переделать метод класса-родителя.

Представим, что мы наследуем класс, но нам не подходит то, что мы унаследовали.

Допустим, мы хотим, чтобы определённый метод работал не так, как в родителе. Чтобы переопределить метод класса-родителя, пишем над ним **@Override**

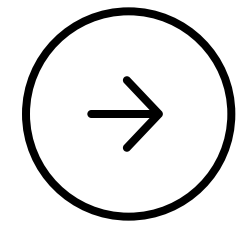
Правила наследования

→ Правило 4. Вызываем методы родителя через ключевое слово `super`.

Представим, что вы хотите изменить метод родительского класса совсем чуть-чуть — буквально дописать пару строк. Тогда в своём методе мы можем вызвать родительский метод с помощью ключевого слова **super**

```
public class Singer extends Person {  
    public void sing(String verse) {  
        System.out.println("Я пою: " + verse);  
    }  
    @Override  
    public void happyBirthday(){  
        super.happyBirthday();  
        System.out.println("Я пою себе с др!");  
    }  
}
```

Правила наследования



Правило 5. Запрещаем наследование.

Если вы не хотите, чтобы кто-то наследовал ваш класс, поставьте перед ним модификатор **final**. Например:

```
final class Person {  
}
```

```
class Manager extends Person {  
}
```

Получим ошибку

Наследование полей

Важно помнить, что объявление одноимённых полей в базовом и производном классах приводит не к смене версий поля, а к созданию новых полей с одинаковыми именами. Таким образом, в производном классе будут два поля одинаковыми названиями.

Если мы создадим метод с использованием этого поля в производном классе, то Java будет понимать, что мы обращаемся именно к полю производного класса, а не базового

Инкапсуляция при наследовании

Если мы сделаем поля базового класса приватными, то мы не сможем к ним обратиться в производном классе. Чтобы этого избежать, вместо идентификатора **private** необходимо использовать **protected**:

```
public class Person {  
    protected String name;  
    protected int age;  
}
```

Если мы не хотим, чтобы от нашего класса наследовались, то ставим перед словом **class** слово **final**:

```
public final class Person {}
```

Наследование конструкторов



Конструкторы при наследовании

Если в базовом классе определены конструкторы, то в конструкторе производного класса необходимо вызвать один из конструкторов базового класса с помощью ключевого слова **super**.

Например, класс **Person** имеет конструктор, который принимает два параметра. Поэтому в классе **Singer** в конструкторе нужно вызвать конструктор класса **Person**. После слова `super` в скобках идёт перечисление передаваемых аргументов.

При этом вызов конструктора базового класса должен идти в самом начале, в конструкторе производного класса

Конструкторы при наследовании

Пример:

```
public class Person {  
    protected String name;  
    protected int age;  
    public Person (String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
class Singer extends Person {  
    public int rating;  
    public Singer (String name, int age, int rating) {  
        super(name, age);  
        this.rating = rating;  
    }  
}
```


Конструкторы при наследовании

Вызов конструктора суперкласса должен быть первой инструкцией в конструкторе дочернего класса. Можно вызвать конструктор суперкласса без параметров (конструктор по умолчанию):

```
class Singer extends Person {  
    public int rating;  
    public Singer (String name, int age, int rating) {  
        super();  
        this.rating = rating  
    }  
}
```

Конструкторы при наследовании

Если вы не вставили ни одного явного вызова конструктора родительского класса, то компилятор Java автоматически добавит вызов конструктора родительского класса без параметров (конструктор по умолчанию).

Если конструктор родительского класса без параметров недоступен из-за модификатора доступа или конструктора без параметров нет в родительском классе, то возникнет ошибка компиляции.

При создании экземпляра любого объекта происходит цепочка вызовов конструкторов: от конструктора создаваемого объекта до конструктора класса Object. Это называется цепочкой вызова конструкторов (constructor chaining)